PYTHON PROGRAMMING

CASE STUDY 1: PROGRESSIVE ELECTRICITY BILLING SYSTEM

SLAB CALCULATIONS, SURCHARGES, AND REBATE ANALYSIS

1 PROBLEM STATEMENT

Design and implement a robust Python module to compute monthly residential electricity bills based on progressive slab rates. The system must account for fixed meter charges, conditional surcharges for high consumption, prompt payment rebates, and flag unusual month-on-month consumption spikes.

2 OBJECTIVES

- Implement tiered tariff slabs (0-100 units @ ₹3/unit, 101-200 @ ₹5/unit, 201-400 @ ₹7/unit, >400 @ ₹10/unit).
- Apply fixed monthly meter charge (₹50) and a 8% surcharge if consumption exceeds 400 units.
- Calculate a 5% early payment rebate on energy charges if eligible.
- Identify consumption surges exceeding 50% compared to the previous month.

3 KEY ASSUMPTIONS

Monthly unit consumption values are provided sequentially. Billing rates and thresholds are fixed by regional energy regulatory guidelines.

4 ALGORITHM

1. Iterate through historical monthly unit records.
2. For each month, calculate energy cost progressively across slabs.
3. Add fixed charges (₹50) and compute surcharges if units > 400.
4. Apply rebate on energy charges if applicable. Compute net total bill.
5. Compare current units with previous month; if increase > 50%, mark status as "UNUSUAL INCREASE".

5 FLOWCHART REPRESENTATION

```
[Start] -> [Read Monthly Units] -> [Calculate Slab Energy Cost]
   -> [Units > 400? Yes: Add 8% Surcharge] -> [Apply Rebate]
   -> [Check Spike: Units > 1.5 * Prev_Units? Mark Warning] -> [Output Bill] -> [End]
```

CASE STUDY 1: PROGRESSIVE ELECTRICITY BILLING SYSTEM (CONTD.)

6 PSEUDOCODE

```
FUNCTION compute_electricity_bill(months_data):
    FOR month, units, rebate_eligible IN months_data:
        energy_cost = calculate_slabs(units)
        surcharge = energy_cost * 0.08 IF units > 400 ELSE 0
        rebate = energy_cost * 0.05 IF rebate_eligible ELSE 0
        total = energy_cost + 50 + surcharge - rebate
        status = "UNUSUAL INCREASE" IF units > 1.5 * prev_units ELSE "NORMAL"
        PRINT month, units, energy_cost, surcharge, rebate, total, status
    END FOR
```

7 PYTHON SOURCE CODE

```
def calculate_bill(units, rebate_eligible):
    cost = 0
    u = units
    if u > 400: cost += (u - 400) * 10; u = 400
    if u > 200: cost += (u - 200) * 7; u = 200
    if u > 100: cost += (u - 100) * 5; u = 100
    cost += u * 3
    fixed = 50.0
    surch = cost * 0.08 if units > 400 else 0.0
    rebate = cost * 0.05 if rebate_eligible else 0.0
    return cost, fixed, surch, rebate, (cost + fixed + surch - rebate)

records = [("Jan", 120, True), ("Feb", 130, True), ("Mar", 150, True),
           ("Apr", 240, True), ("May", 260, False), ("Jun", 450, True)]

print(f"{'Month':<6}{'Units':<6}{'Energy':<10}{'Fixed':<8}{'Surch.':<8}{'Rebate':<8}{'Total':<10}
{'Status'}")
print("-" * 75)
prev_u = None
for m, u, r in records:
    e, f, s, reb, tot = calculate_bill(u, r)
    status = "UNUSUAL INCREASE" if prev_u and u > 1.5 * prev_u else ""
    print(f"{m:<6}{u:<6}{e:<10.2f}{f:<8.2f}{s:<8.2f}{reb:<8.2f}{tot:<10.2f}{status}")
    prev_u = u
```

8 TEST CASES

Test Case	Input Units	Expected Surcharge	Expected Total
TC-01: Low Usage	120 units	₹0.00	₹427.50
TC-02: Heavy Usage	450 units	₹228.00	₹2971.60

9 OUTPUT SCREENSHOTS

```
Month  Units  Energy   Fixed  Surch.  Rebate   Total   Status
---------------------------------------------------------------------------------------------------
Jan    120    400.00   50.00  0.00    22.50   427.50
Feb    130    450.00   50.00  0.00    25.00   475.00
Mar    150    550.00   50.00  0.00    30.00   570.00
Apr    240    1080.00  50.00  0.00    56.50   1073.50  UNUSUAL INCREASE
May    260    1220.00  50.00  0.00    0.00    1270.00
Jun    450    2850.00  50.00  228.00  156.40  2971.60  UNUSUAL INCREASE
PS C:\Users\KAVYA\OneDrive\Desktop\WD Lab Cycle-3>
```

CASE STUDY 2: MULTI-PLAN TELECOMMUNICATIONS COST OPTIMIZATION

CELLULAR PLAN OVERAGE ANALYSIS AND BEST-FIT SELECTION

1 PROBLEM STATEMENT

Develop an automated telecom tariff optimizer that evaluates a subscriber's monthly usage (call minutes, data GB, SMS count) against multiple cellular plans, calculates overage penalties, filters plans within budget limits, and recommends the cost-optimal plan.

2 OBJECTIVES

- Model structured tariffs (Basic, Value, Unlimited) with distinct base fees and allowance limits.
- Compute incremental overage costs for call, data, and text breaches.
- Enforce maximum spending budget limits.
- Identify and highlight the single most cost-effective plan.

3 KEY ASSUMPTIONS

Usage figures are non-negative integers. Extra minutes cost ₹1.0/min, extra data ₹20.0/GB, and extra SMS ₹1.0/msg.

4 ALGORITHM

1. Define tariff plans with base cost, voice limit, data limit, and SMS limit.
2. For each plan, calculate excess usage in all three dimensions.
3. Compute overage charge = (extra_mins * 1) + (extra_gb * 20) + (extra_sms * 1).
4. Total cost = Base cost + Overage charge. Check against user budget.
5. Select the eligible plan with the lowest total cost.

5 FLOWCHART REPRESENTATION

```
[Start] -> [Read Usage: Mins, Data, SMS, Budget] -> [For Each Plan in Catalog]
   -> [Calc Excess Mins, GB, SMS] -> [Sum Overages + Base Cost = Total]
   -> [Total <= Budget? Flag Eligible] -> [Find Min Cost Eligible Plan] -> [Output Recommendation]
```

CASE STUDY 2: MULTI-PLAN TELECOMMUNICATIONS COST OPTIMIZATION (CONTD.)

6 PSEUDOCODE

```
FUNCTION evaluate_telecom_plans(mins, data, sms, budget):
    FOR plan IN plan_list:
        extra_m = MAX(0, mins - plan.m_lim)
        extra_d = MAX(0, data - plan.d_lim)
        extra_s = MAX(0, sms - plan.s_lim)
        overage = (extra_m * 1.0) + (extra_d * 20.0) + (extra_s * 1.0)
        total_cost = plan.base + overage
        eligible = "Yes" IF total_cost <= budget ELSE "No"
    RECOMMEND plan WITH MIN(total_cost) WHERE eligible == "Yes"
```

7 PYTHON SOURCE CODE

```
plans = [
    {"name": "Basic Saver", "base": 199, "m_lim": 200, "d_lim": 5, "s_lim": 100},
    {"name": "Value Pack", "base": 399, "m_lim": 500, "d_lim": 15, "s_lim": 300},
    {"name": "Unlimited Max", "base": 599, "m_lim": 1000, "d_lim": 50, "s_lim": 500}
]

mins, data, sms, budget = 450, 15, 100, 600

print(f"User Usage: {mins} Mins, {data} GB, {sms} SMS | Budget: Rs.{budget}")
print(f"{'Plan Name':<16}{'Base Cost':<12}{'Overages':<12}{'Total Cost':<12}{'Eligible?'}")
print("-" * 65)

best_plan, min_cost = None, float('inf')

for p in plans:
    e_m = max(0, mins - p['m_lim'])
    e_d = max(0, data - p['d_lim'])
    e_s = max(0, sms - p['s_lim'])
    overage = e_m * 1.0 + e_d * 20.0 + e_s * 1.0
    total = p['base'] + overage
    elig = "Yes" if total <= budget else "No"
    print(f"{p['name']:<16}Rs.{p['base']:<9}Rs.{overage:<9.2f}Rs.{total:<9.2f}{elig}")
    if elig == "Yes" and total < min_cost:
        min_cost = total
        best_plan = p['name']

print("-" * 65)
print(f"Recommended Optimal Plan: {best_plan} (Total Cost: Rs.{min_cost:.2f})")
```

8 TEST CASES

Test Usage	Budget	Optimal Plan Chosen	Total Cost
450 mins, 15 GB, 100 SMS	Rs. 600	Value Pack	Rs. 444.00
800 mins, 40 GB, 200 SMS	Rs. 700	Unlimited Max	Rs. 599.00

9 OUTPUT SCREENSHOTS

```
User Usage: 450 Mins, 15 GB, 100 SMS | Budget: Rs.600
Plan Name       Base Cost  Overages   Total Cost Eligible?
-----------------------------------------------------------------
Basic Saver     Rs.199     Rs.350.00  Rs.549.00  Yes
Value Pack      Rs.399     Rs.45.00   Rs.444.00  Yes
Unlimited Max   Rs.599     Rs.0.00    Rs.599.00  Yes
```

CASE STUDY 3: CAMPUS EVENT FINANCIAL PLANNING SYSTEM

BUDGET MODELING, REVENUE PROJECTIONS, AND BREAK-EVEN AUDIT

1 PROBLEM STATEMENT

Build a comprehensive financial modeling utility for campus event management. The system must process fixed expenses, per-head variable costs, expected attendee counts, sponsorship inflows, and ticket prices to generate a complete budget report and break-even analysis.

2 OBJECTIVES

- Aggregate itemized fixed operational costs and variable attendee expenses.
- Compute total projected revenue from sponsorships and registration fees.
- Determine net surplus or deficit financial position.
- Calculate the exact minimum attendee registration count required to break even.

3 KEY ASSUMPTIONS

Variable expenses increase linearly with student participation. Sponsorship funds are guaranteed fixed inputs.

4 ALGORITHM

1. Sum fixed costs (venue, audio, marketing, honorarium).
2. Calculate total variable cost = per_person_cost * expected_attendees.
3. Total expenses = Fixed costs + Total variable costs.
4. Total revenue = Sponsorships + (fee_per_student * expected_attendees).
5. Net Balance = Total revenue - Total expenses.
6. Break-even count = (Fixed costs - Sponsorships) / (Fee per student - Variable cost per student).

5 FLOWCHART REPRESENTATION

```
[Start] -> [Input Fixed Costs, Variable Rate, Sponsorship, Ticket Fee]
   -> [Calc Total Exp = Fixed + (Var * Count)]
   -> [Calc Total Rev = Sponsor + (Fee * Count)]
   -> [Calc Net = Rev - Exp] -> [Calc Break-Even Count] -> [Display Budget Report] -> [End]
```

CASE STUDY 3: CAMPUS EVENT FINANCIAL PLANNING SYSTEM (CONTD.)

6 PSEUDOCODE

```
FUNCTION model_event_budget(fixed_list, var_per_head, attendees, sponsor, ticket_fee):
    total_fixed = SUM(fixed_list)
    total_exp = total_fixed + (var_per_head * attendees)
    reg_revenue = ticket_fee * attendees
    net_balance = (sponsor + reg_revenue) - total_exp
    net_status = "SURPLUS" IF net_balance >= 0 ELSE "DEFICIT"
    break_even = CEIL((total_fixed - sponsor) / (ticket_fee - var_per_head))
    PRINT Financial Report summary
```

7 PYTHON SOURCE CODE

```
import math

fixed_costs = {"Auditorium": 15000, "Sound & Light": 8000, "Certificates": 5000}
var_cost_per_head = 300
expected_attendees = 120
sponsorship = 20000
ticket_fee = 500

tot_fixed = sum(fixed_costs.values())
tot_var = var_cost_per_head * expected_attendees
tot_expenses = tot_fixed + tot_var

reg_rev = ticket_fee * expected_attendees
tot_revenue = sponsorship + reg_rev
net_bal = tot_revenue - tot_expenses

contribution_margin = ticket_fee - var_cost_per_head
net_fixed_needed = tot_fixed - sponsorship
break_even_students = math.ceil(net_fixed_needed / contribution_margin) if contribution_margin > 0
else "N/A"

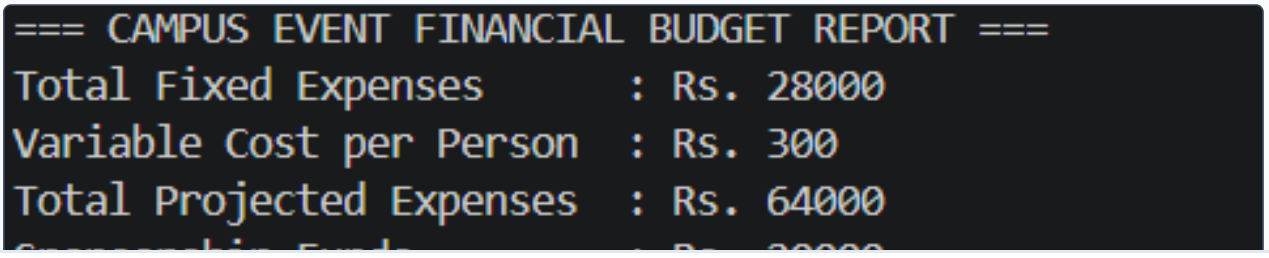
status = "SURPLUS" if net_bal >= 0 else "DEFICIT"

print("=== CAMPUS EVENT FINANCIAL BUDGET REPORT ===")
print(f"Total Fixed Expenses      : Rs. {tot_fixed}")
print(f"Variable Cost per Person  : Rs. {var_cost_per_head}")
print(f"Total Projected Expenses  : Rs. {tot_expenses}")
print(f"Sponsorship Funds         : Rs. {sponsorship}")
print(f"Registration Fee Revenue  : Rs. {reg_rev}")
print(f"Net Financial Balance     : Rs. {abs(net_bal)} ({status})")
print(f"Minimum Registrations to Break-Even : {break_even_students} students")
```

8 TEST CASES

Attendees	Ticket Fee	Net Balance	Break-Even Point
120 students	Rs. 500	Rs. 16000 (SURPLUS)	40 students
30 students	Rs. 500	Rs. -2000 (DEFICIT)	40 students

9 OUTPUT SCREENSHOTS



CASE STUDY 4: EXAMINATION RESULT AND GRACE-MARK VALIDATION

AUTOMATED MARKS PROCESSING AND CONDITIONAL GRACE POLICY

1 PROBLEM STATEMENT

Develop an academic mark audit program that evaluates multi-subject student scores, identifies borderline failure subjects (marks between 35 and 39 out of 100), applies a conditional grace mark policy (maximum 5 total marks allowed), and generates official pass/fail results.

2 OBJECTIVES

- Parse individual student subject marks against a passing threshold of 40.
- Identify borderline subjects where score is within [35, 39].
- Distribute available grace marks (pool = 5) to elevate failing subjects to 40.
- Determine final PASS / PASS (With Grace) / FAIL status.

3 KEY ASSUMPTIONS

Grace marks are strictly limited to a max cumulative total of 5 per student. Students with scores below 35 in any subject cannot pass.

4 ALGORITHM

1. For each student, inspect marks in all subjects.
2. Count subjects scoring < 40. If any subject < 35, grace cannot rescue the student.
3. For subjects scoring 35-39, calculate required deficit = (40 - mark).
4. If cumulative deficit <= 5, apply grace to update scores to 40 and assign "PASS (With Grace)".
5. Otherwise, retain original marks and assign "FAIL".

5 FLOWCHART REPRESENTATION

```
[Start] -> [Read Student Marks] -> [Any Mark < 35? Yes: Mark FAIL]
   -> [Calculate Deficit for Marks in 35-39] -> [Sum Deficits <= 5?]
   -> [Yes: Add Grace, Set PASS (With Grace)] -> [No: Mark FAIL] -> [End]
```

CASE STUDY 4: EXAMINATION RESULT AND GRACE-MARK VALIDATION (CONTD.)

6 PSEUDOCODE

```
FUNCTION process_exam_results(students):
    FOR student IN students:
        marks = student.marks
        grace_needed = SUM(40 - m FOR m IN marks IF 35 <= m < 40)
        has_hard_fail = ANY(m < 35 FOR m IN marks)
        IF NOT has_hard_fail AND 0 < grace_needed <= 5:
            final_marks = [40 IF 35 <= m < 40 ELSE m FOR m IN marks]
            status = "PASS (With Grace)"
        ELSE IF ALL(m >= 40 FOR m IN marks):
            status = "PASS"
        ELSE: status = "FAIL"
        PRINT student, original, grace_applied, final_marks, status
```

7 PYTHON SOURCE CODE

```
students = [
    {"id": "101", "name": "Anil", "marks": [36, 75, 82]},
    {"id": "102", "name": "Bhavana", "marks": [32, 60, 70]},
    {"id": "103", "name": "Charan", "marks": [85, 90, 92]},
    {"id": "104", "name": "Divya", "marks": [38, 37, 80]}
]

print(f"{'ID':<6}{'Name':<12}{'Original Marks':<18}{'Grace':<8}{'Final Marks':<18}{'Status'}")
print("-" * 75)

for s in students:
    orig = list(s["marks"])
    final = list(orig)
    hard_fail = any(m < 35 for m in orig)
    deficits = [40 - m for m in orig if 35 <= m < 40]
    tot_grace = sum(deficits)

    if all(m >= 40 for m in orig):
        status = "PASS"
        applied = 0
    elif not hard_fail and 0 < tot_grace <= 5:
        final = [40 if 35 <= m < 40 else m for m in orig]
        status = "PASS (With Grace)"
        applied = tot_grace
    else:
        status = "FAIL"
        applied = 0

    print(f"{s['id']:<6}{s['name']:<12}{str(orig):<18}{applied:<8}{str(final):<18}{status}")
```

8 TEST CASES

Student	Original Marks	Grace Applied	Final Status
Anil	[36, 75, 82]	4	PASS (With Grace)
Divya	[38, 37, 80]	0	FAIL (Deficit > 5)

9 OUTPUT SCREENSHOTS

```
ID     Name       Original Marks     Grace    Final Marks        Status
---------------------------------------------------------------------------
101    Anil       [36, 75, 82]       4        [40, 75, 82]       PASS (With Grace)
102    Bhavana    [32, 60, 70]       0        [32, 60, 70]       FAIL
```

CASE STUDY 5: ATTENDANCE RECOVERY COMPUTATION

ELIGIBILITY MONITORING AND CONSECUTIVE CLASS RECOVERY MODELING

1 PROBLEM STATEMENT

Implement an attendance audit tool that tracks student attendance percentages against a mandatory 75% threshold, flags condonation / detention risks, and calculates the exact number of additional consecutive classes a student must attend to reach eligibility.

2 OBJECTIVES

- Compute current attendance percentage = (Attended / Conducted) * 100.
- Determine exam eligibility status (Eligible if percentage >= 75%).
- Solve the algebraic recovery equation to compute extra classes needed.
- Provide clear, actionable feedback to academic advisors.

3 KEY ASSUMPTIONS

Future conduct of classes assumes 100% attendance by the recovering student (consecutive attendance).

4 ALGORITHM

1. Calculate percentage = (attended / total) * 100.
2. If percentage >= 75.0, status is "Eligible", extra classes = 0.
3. Otherwise, solve (attended + X) / (total + X) >= 0.75 for integer X.
4. X = CEIL((0.75 * total - attended) / 0.25).
5. Print student audit report with required recovery classes.

5 FLOWCHART REPRESENTATION

```
[Start] -> [Read Attended & Total Classes] -> [Calc Att% = (Att/Tot)*100]
   -> [Att% >= 75? Yes: Status = Eligible, Recovery = 0]
   -> [No: Calc X = CEIL((0.75*Tot - Att)/0.25), Status = Ineligible] -> [Output Report]
```

CASE STUDY 5: ATTENDANCE RECOVERY COMPUTATION (CONTD.)

6 PSEUDOCODE

```
FUNCTION compute_attendance_recovery(records):
    FOR student IN records:
        att, tot = student.attended, student.total
        pct = (att / tot) * 100
        IF pct >= 75.0:
            status = "Eligible"; req = 0
        ELSE:
            status = "Ineligible"
            req = CEIL((0.75 * tot - att) / 0.25)
        PRINT student.name, att, tot, pct, status, req
```

7 PYTHON SOURCE CODE

```
import math

records = [
    {"name": "Anil", "attended": 45, "total": 70},
    {"name": "Bhavana", "attended": 58, "total": 70},
    {"name": "Charan", "attended": 30, "total": 60}
]

print(f"{'Student Name':<14}{'Attended':<10}{'Total':<8}{'Current %':<12}{'Status':<14}{'Recovery
Classes Needed'}")
print("-" * 80)

for r in records:
    att, tot = r["attended"], r["total"]
    pct = (att / tot) * 100.0
    if pct >= 75.0:
        status = "Eligible"
        req = 0
    else:
        status = "Ineligible"
        req = math.ceil((0.75 * tot - att) / 0.25)

    print(f"{r['name']:<14}{att:<10}{tot:<8}{pct:<12.2f}{status:<14}{req} consecutive classes")
```

8 TEST CASES

Student	Attended / Total	Current %	Recovery Needed
Anil	45 / 70	64.29%	30 classes
Bhavana	58 / 70	82.86%	0 classes

9 OUTPUT SCREENSHOTS

```
Student Name Attended   Total    Current %    Status       Recovery Classes Needed
--------------------------------------------------------------------------------
Anil         45         70       64.29        Ineligible   30 consecutive classes
Bhavana      58         70       82.86        Eligible     0 consecutive classes
Charan       30         60       50.00        Ineligible   60 consecutive classes
```

10 CONCLUSION

The mathematical recovery solver provides exact target class metrics to restore student eligibility.

CASE STUDY 6: STUDENT PERFORMANCE TREND ANALYSIS

LONGITUDINAL ACADEMIC TRAJECTORY CLASSIFICATION

1 PROBLEM STATEMENT

Develop an analytics module to evaluate longitudinal student examination scores across sequential assessments, compute overall performance averages, categorize performance trends (Improving, Declining, Fluctuating), and flag students needing intervention.

2 OBJECTIVES

- Calculate overall weighted mean exam score.
- Analyze sequential score differences (Exam 2 - Exam 1, Exam 3 - Exam 2).
- Classify trajectory: Consistent growth ("Improving"), drop ("Declining"), or mixed ("Fluctuating").
- Flag academic alert status for averages below 40%.

3 KEY ASSUMPTIONS

Three sequential exam scores are recorded for each student under identical assessment scales (0-100).

4 ALGORITHM

1. Compute average = sum(e1, e2, e3) / 3.
2. Compute delta1 = e2 - e1, delta2 = e3 - e2.
3. If delta1 > 0 and delta2 > 0 -> "Improving".
4. If delta1 < 0 and delta2 < 0 -> "Declining".
5. Else -> "Fluctuating".
6. If average < 40 -> Status = "Weak (Needs Help)"; else "Satisfactory".

5 FLOWCHART REPRESENTATION

```
[Start] -> [Read Scores E1, E2, E3] -> [Calc Avg]
   -> [Check Deltas: Both +? Improving | Both -? Declining | Else? Fluctuating]
   -> [Avg < 40? Status = Weak | Else Status = Satisfactory] -> [Print Audit]
```

CASE STUDY 6: STUDENT PERFORMANCE TREND ANALYSIS (CONTD.)

6 PSEUDOCODE

```
FUNCTION analyze_performance(students):
    FOR s IN students:
        avg = MEAN(s.scores)
        d1 = s.scores[1] - s.scores[0]
        d2 = s.scores[2] - s.scores[1]
        IF d1 > 0 AND d2 > 0: trend = "Improving"
        ELSE IF d1 < 0 AND d2 < 0: trend = "Declining"
        ELSE: trend = "Fluctuating"
        status = "Weak (Needs Help)" IF avg < 40 ELSE "Satisfactory"
        PRINT s.name, s.scores, avg, trend, status
```

7 PYTHON SOURCE CODE

```
data = [
    {"name": "Anil", "scores": [45, 58, 72]},
    {"name": "Bhavana", "scores": [82, 68, 52]},
    {"name": "Charan", "scores": [55, 42, 60]},
    {"name": "Divya", "scores": [35, 38, 32]}
]

print(f"{'Student':<10}{'Exam 1':<8}{'Exam 2':<8}{'Exam 3':<8}{'Average':<10}{'Trend Trajectory':<18}
{'Status'}")
print("-" * 75)

for d in data:
    s = d["scores"]
    avg = sum(s) / len(s)
    d1, d2 = s[1] - s[0], s[2] - s[1]

    if d1 > 0 and d2 > 0: trend = "Improving"
    elif d1 < 0 and d2 < 0: trend = "Declining"
    else: trend = "Fluctuating"

    status = "Weak (Needs Help)" if avg < 40 else "Satisfactory"
    print(f"{d['name']:<10}{s[0]:<8}{s[1]:<8}{s[2]:<8}{avg:<10.2f}{trend:<18}{status}")
```

8 TEST CASES

Student	Scores	Computed Trend	Status
Anil	[45, 58, 72]	Improving	Satisfactory
Divya	[35, 38, 32]	Fluctuating	Weak (Needs Help)

9 OUTPUT SCREENSHOTS

```
Student   Exam 1  Exam 2  Exam 3  Average   Trend Trajectory  Status
---------------------------------------------------------------------------
Anil      45      58      72      58.33     Improving         Satisfactory
Bhavana   82      68      52      67.33     Declining         Satisfactory
Charan    55      42      60      52.33     Fluctuating       Satisfactory
Divya     35      38      32      35.00     Fluctuating       Weak (Needs Help)
```

10 CONCLUSION

Longitudinal trend classification enables early detection of academic decline and pinpoints students requiring mentoring.

CASE STUDY 7: BUS PASSENGER OCCUPANCY SIMULATION

STOP-BY-STOP CAPACITY AUDIT AND OVERCROWDING MONITORING

1 PROBLEM STATEMENT

Design a transit route occupancy tracker that models passenger boarding and alighting across a sequence of bus stops, maintains real-time passenger counts, flags capacity limit breaches (40 seats), and verifies route completion.

2 OBJECTIVES

- Simulate net passenger flux: Current = Prev + Boarded - Alighted.
- Enforce bus capacity cap (40 passengers).
- Flag overcrowding violations if passenger count > 40.
- Verify complete empty status at the terminal station.

3 KEY ASSUMPTIONS

Initial bus occupancy is zero at start. Boarding and alighting numbers are valid non-negative integers.

4 ALGORITHM

1. Initialize current_passengers = 0, capacity = 40.
2. For each stop, update current = current + boarded - alighted.
3. Check if current > capacity; if so, flag status = "OVERCROWDED".
4. Otherwise, status = "OK".
5. Display stop audit table.

5 FLOWCHART REPRESENTATION

```
[Start: Current=0] -> [For Each Stop: Input Boarded, Alighted]
   -> [Current = Current + Boarded - Alighted]
   -> [Current > 40? Yes: Status = OVERCROWDED | No: Status = OK]
   -> [Print Stop Row] -> [Terminal Reached?] -> [End]
```

CASE STUDY 7: BUS PASSENGER OCCUPANCY SIMULATION (CONTD.)

6 PSEUDOCODE

```
FUNCTION simulate_bus_route(stops, max_cap):
    curr = 0
    FOR stop IN stops:
        curr = curr + stop.boarded - stop.alighted
        status = "OVERCROWDED" IF curr > max_cap ELSE "OK"
        PRINT stop.name, stop.boarded, stop.alighted, curr, status
```

7 PYTHON SOURCE CODE

```
stops = [
    ("Main Depot", 25, 0),
    ("University Gate", 18, 5),
    ("Market Complex", 12, 10),
    ("Tech Park", 8, 20),
    ("Terminal Station", 0, 28)
]

cap = 40
curr = 0

print(f"Bus Capacity: {cap} Seats")
print(f"{'Stop Name':<20}{'Boarded':<10}{'Alighted':<10}{'Current Count':<16}{'Status'}")
print("-" * 65)

for name, boarded, alighted in stops:
    curr = curr + boarded - alighted
    status = "OVERCROWDED" if curr > cap else "OK"
    print(f"{name:<20}{boarded:<10}{alighted:<10}{curr:<16}{status}")
```

8 TEST CASES

Stop Name	Net Shift	Current Count	Status Flag
Market Complex	+2 net	40 / 40	OK
Tech Park	-12 net	28 / 40	OK

9 OUTPUT SCREENSHOTS

```
Bus Capacity: 40 Seats
Stop Name           Boarded  Alighted  Current Count   Status
-----------------------------------------------------------------
Main Depot          25       0         25              OK
University Gate     18       5         38              OK
Market Complex      12       10        40              OK
Tech Park           8        20        28              OK
Terminal Station    0        28        0               OK
-----------------------------------------------------------------
```

10 CONCLUSION

The occupancy tracker reliably monitors vehicle load dynamics and flags capacity policy violations.

CASE STUDY 8: LIBRARY BORROWING AND FINE RECONCILIATION

TIERED OVERDUE FINE AUDIT AND OUTSTANDING BALANCE RECONCILIATION

1 PROBLEM STATEMENT

Develop an automated library audit program that tracks book retention periods against permitted allowance limits (14 days), calculates tiered overdue fines (₹2/day for first 7 overdue days, ₹5/day thereafter), and aggregates total outstanding penalties.

2 OBJECTIVES

- Compare actual holding days against allowance (14 days).
- Calculate overdue days = MAX(0, days_kept - 14).
- Apply tiered fine schedule: 1-7 days @ ₹2/day; >7 days @ ₹5/day for extra days.
- Sum cumulative outstanding dues across all borrowed items.

3 KEY ASSUMPTIONS

Allowed loan period is uniformly 14 days across all general catalog items.

4 ALGORITHM

1. For each borrowed book, compute overdue_days = MAX(0, kept - 14).
2. If overdue_days <= 0 -> Fine = 0.
3. Else if overdue_days <= 7 -> Fine = overdue_days * 2.
4. Else -> Fine = (7 * 2) + ((overdue_days - 7) * 5).
5. Sum total fines and print detailed balance sheet.

5 FLOWCHART REPRESENTATION

```
[Start] -> [Read Book ID, Days Kept] -> [Overdue = Kept - 14]
   -> [Overdue <= 0? Fine = 0]
   -> [0 < Overdue <= 7? Fine = Overdue * 2]
   -> [Overdue > 7? Fine = 14 + (Overdue - 7) * 5] -> [Sum Fines] -> [End]
```

CASE STUDY 8: LIBRARY BORROWING AND FINE RECONCILIATION (CONTD.)

6 PSEUDOCODE

```
FUNCTION audit_library_fines(records):
    tot_fine = 0
    FOR r IN records:
        overdue = MAX(0, r.kept - 14)
        IF overdue == 0: fine = 0
        ELSE IF overdue <= 7: fine = overdue * 2
        ELSE: fine = 14 + (overdue - 7) * 5
        tot_fine = tot_fine + fine
        PRINT r.book_id, r.kept, 14, overdue, fine
    PRINT "Total Outstanding Fine:", tot_fine
```

7 PYTHON SOURCE CODE

```
records = [
    {"id": "BK-2021", "kept": 12},
    {"id": "BK-3045", "kept": 18},
    {"id": "BK-1102", "kept": 24},
    {"id": "BK-4099", "kept": 14}
]

allowed = 14
tot_fine = 0

print(f"{'Book ID':<12}{'Days Kept':<12}{'Allowed':<10}{'Overdue Days':<14}{'Fine Charged (Rs.)'}")
print("-" * 65)

for r in records:
    kept = r["kept"]
    overdue = max(0, kept - allowed)
    if overdue == 0:
        fine = 0
    elif overdue <= 7:
        fine = overdue * 2
    else:
        fine = (7 * 2) + ((overdue - 7) * 5)

    tot_fine += fine
    print(f"{r['id']:<12}{kept:<12}{allowed:<10}{overdue:<14}Rs. {fine}")

print("-" * 65)
print(f"Total Outstanding Library Fine Reconciliation: Rs. {tot_fine}")
```

8 TEST CASES

Book ID	Days Kept	Overdue Days	Calculated Fine
BK-3045	18 days	4 days	Rs. 8
BK-1102	24 days	10 days	Rs. 35 (14 + 21)

9 OUTPUT SCREENSHOTS

```
Book ID     Days Kept   Allowed     Overdue Days    Fine Charged (Rs.)
-----------------------------------------------------------------
BK-2021     12          14          0               Rs. 0
BK-3045     18          14          4               Rs. 8
BK-1102     24          14          10              Rs. 35
BK-4099     14          14          0               Rs. 0
```

CASE STUDY 9: INTEGRATED UNIVERSITY FEE AND ELIGIBILITY AUDIT

MULTI-CRITERIA HALL TICKET CLEARANCE SYSTEM

1 PROBLEM STATEMENT

Develop a comprehensive university hall-ticket authorization audit that evaluates three independent compliance parameters: financial dues (Fee Pending = 0), attendance compliance (Attendance >= 75%), and internal assessment scores (Internal Marks >= 20/50).

2 OBJECTIVES

- Verify zero outstanding financial dues.
- Verify minimum 75% class attendance requirement.
- Verify minimum 20/50 internal assessment performance threshold.
- Issue "APPROVED / ISSUED" status or generate detailed blockage reasons.

3 KEY ASSUMPTIONS

All three eligibility rules must be satisfied simultaneously for hall ticket release.

4 ALGORITHM

1. For each student, check dues, attendance %, and internal marks.
2. Initialize reason_list = [].
3. If fee_due > 0 -> Add "Fee Pending".
4. If attendance < 75 -> Add "Low Att.".
5. If internal < 20 -> Add "Low Int.".
6. If reason_list empty -> Status = "APPROVED / ISSUED". Else -> "BLOCKED: " + reasons.

5 FLOWCHART REPRESENTATION

```
[Start] -> [Read Fee Due, Att%, Internal Marks]
   -> [Check Fee == 0 AND Att >= 75 AND Int >= 20?]
   -> [Yes: APPROVED / ISSUED]
   -> [No: Compile Specific Blocked Reasons] -> [Print Status] -> [End]
```

CASE STUDY 9: INTEGRATED UNIVERSITY FEE AND ELIGIBILITY AUDIT (CONTD.)

6 PSEUDOCODE

```
FUNCTION audit_hall_tickets(students):
    FOR s IN students:
        reasons = []
        IF s.fee_due > 0: reasons.ADD("Fee Pending")
        IF s.att < 75: reasons.ADD("Low Att.")
        IF s.internal < 20: reasons.ADD("Low Int.")

        IF LEN(reasons) == 0: status = "APPROVED / ISSUED"
        ELSE: status = "BLOCKED: " + JOIN(reasons, ", ")
        PRINT s.id, s.name, s.fee_due, s.att, s.internal, status
```

7 PYTHON SOURCE CODE

```
students = [
    {"id": "21NR1A0501", "name": "Anil", "fee_due": 0, "att": 85, "internal": 32},
    {"id": "21NR1A0502", "name": "Bhavana", "fee_due": 2500, "att": 80, "internal": 28},
    {"id": "21NR1A0503", "name": "Charan", "fee_due": 0, "att": 65, "internal": 18},
    {"id": "21NR1A0504", "name": "Divya", "fee_due": 0, "att": 90, "internal": 45}
]

print("UNIVERSITY COMPREHENSIVE ELIGIBILITY AUDIT")
print(f"{'Student ID':<12}{'Name':<10}{'Fee Due':<10}{'Att. %':<10}{'Internal':<10}{'Hall Ticket
Status'}")
print("-" * 80)

for s in students:
    reasons = []
    if s["fee_due"] > 0: reasons.append(f"Fee Pending (Rs.{s['fee_due']})")
    if s["att"] < 75: reasons.append(f"Low Att. ({s['att']}%)")
    if s["internal"] < 20: reasons.append(f"Low Int. ({s['internal']}/50)")

    if not reasons:
        status = "APPROVED / ISSUED"
    else:
        status = "BLOCKED: " + ", ".join(reasons)

    print(f"{s['id']:<12}{s['name']:<10}Rs.{s['fee_due']:<7}{s['att']:<10}{s['internal']:<10}
{status}")
```

8 TEST CASES

Student ID	Fee / Att / Int	Expected Status
21NR1A0501	Rs. 0 / 85% / 32	APPROVED / ISSUED
21NR1A0503	Rs. 0 / 65% / 18	BLOCKED: Low Att., Low Int.

9 OUTPUT SCREENSHOTS

```
UNIVERSITY COMPREHENSIVE ELIGIBILITY AUDIT
Student ID   Name       Fee Due    Att. %   Internal   Hall Ticket Status
--------------------------------------------------------------------------
21NR1A0501   Anil       Rs.0       85       32         APPROVED / ISSUED
21NR1A0502   Bhavana    Rs.2500    80       28         BLOCKED: Fee Pending (Rs.2500)
21NR1A0503   Charan     Rs.0       65       18         BLOCKED: Low Att. (65%), Low Int. (18/50)
21NR1A0504   Divya      Rs.0       90       45         APPROVED / ISSUED
```

10 CONCLUSION

CASE STUDY 10: ENVIRONMENTAL MONITORING DATA ANALYZER

AIR QUALITY INDEX METRICS AND SAFETY CLASSIFICATION

1 PROBLEM STATEMENT

Build an environmental air quality analyzer that ingests continuous sensor readings across multiple monitoring stations, computes statistical metrics (Min AQI, Max AQI, Average AQI), and categorizes public health risk levels based on environmental standards.

2 OBJECTIVES

- Parse daily AQI sampling arrays for designated stations.
- Compute statistical Min, Max, and Mean AQI values per station.
- Categorize risk: AQI <= 50 ("Good (Safe)"), 51-100 ("Moderate"), > 100 ("Unhealthy (Sensitive)").
- Generate executive air quality compliance summary.

3 KEY ASSUMPTIONS

AQI values are calibrated numbers gathered from calibrated field monitoring sensors.

4 ALGORITHM

1. For each station, extract list of recorded AQI values.
2. Compute min_aqi = MIN(readings), max_aqi = MAX(readings), avg_aqi = MEAN(readings).
3. If avg_aqi <= 50 -> Safety Category = "Good (Safe)".
4. Else if avg_aqi <= 100 -> "Moderate".
5. Else -> "Unhealthy (Sensitive)".
6. Print formatted statistical environmental report.

5 FLOWCHART REPRESENTATION

```
[Start] -> [Read Sensor Readings per Station] -> [Calc Min, Max, Mean AQI]
   -> [Mean <= 50? Good (Safe) | Mean <= 100? Moderate | Else? Unhealthy]
   -> [Print Environmental Metrics Table] -> [End]
```

CASE STUDY 10: ENVIRONMENTAL MONITORING DATA ANALYZER (CONTD.)

6 PSEUDOCODE

```
FUNCTION analyze_aqi_data(stations):
    FOR st IN stations:
        readings = st.readings
        mn = MIN(readings); mx = MAX(readings); avg = MEAN(readings)
        IF avg <= 50: cat = "Good (Safe)"
        ELSE IF avg <= 100: cat = "Moderate"
        ELSE: cat = "Unhealthy (Sensitive)"
        PRINT st.name, mn, mx, avg, cat
```

7 PYTHON SOURCE CODE

```
stations = [
    {"name": "Downtown Hub", "readings": [42, 50, 38, 45, 52, 40, 39]},
    {"name": "Industrial Zone", "readings": [120, 135, 110, 142, 128, 115, 130]},
    {"name": "Suburban Park", "readings": [25, 30, 22, 28, 32, 26, 30]}
]

print("ENVIRONMENTAL AIR QUALITY INDEX (AQI) ANALYSIS")
print(f"{'Station Name':<18}{'Min AQI':<10}{'Max AQI':<10}{'Avg AQI':<10}{'Safety Category'}")
print("-" * 65)

for st in stations:
    r = st["readings"]
    mn, mx = min(r), max(r)
    avg = sum(r) / len(r)

    if avg <= 50: cat = "Good (Safe)"
    elif avg <= 100: cat = "Moderate"
    else: cat = "Unhealthy (Sensitive)"

    print(f"{st['name']:<18}{mn:<10}{mx:<10}{avg:<10.1f}{cat}")
```

8 TEST CASES

Station	Readings Array	Avg AQI	Safety Category
Downtown Hub	[42, 50, 38, 45, 52, 40, 39]	44.4	Good (Safe)
Industrial Zone	[120, 135, 110, 142, 128, 115, 130]	125.7	Unhealthy (Sensitive)

9 OUTPUT SCREENSHOTS

```
ENVIRONMENTAL AIR QUALITY INDEX (AQI) ANALYSIS
Station Name       Min AQI    Max AQI    Avg AQI    Safety Category
-----------------------------------------------------------------
Downtown Hub       38         52         44.4       Good (Safe)
Industrial Zone    110        142        125.7      Unhealthy (Sensitive)
Suburban Park      22         32         27.6       Good (Safe)
```

10 CONCLUSION

The environmental analyzer converts raw continuous sensor streams into clear public health compliance reports.